Google Cloud Platform Project

Cloud Solutions

Tristan De La Borda Baylon
Lycée Guillaume Kroll | BTS-CLC

Contents

Part 1: Multi cloud comparison

In this table we will outline some key differences between the three main providers being AWS, Azure and GCP. This serves as an introductory overview of what this platform in capable of in comparison to its competitors.

Category	AWS	Azure	GCP
Global Infrastructure	Infrastructure is divided into regions which have multiple availability zones within them or datacenters.	Infrastructure also subdivided into regions and AZs which are separate datacenters.	Infrastructure is split into Regions and Zones; regions remains the same as other providers however zones are deployment areas within regions.
Identity & Access Management	AWS Identity and Access Management (IAM)	Azure Active Directory (Azure AD)	Google Cloud Identity and Access Management (IAM)
Virtual Machines Names	Amazon Elastic Compute Cloud (EC2) instances	Azure Virtual Machines	Google Compute Engine (GCE)
Object Storage Format	Amazon Simple Storage Service (S3) buckets	Azure Blob Storage	Google Cloud Storage (GCS)
Virtual Networks	Amazon Virtual Private Cloud (VPC)	Azure Virtual Network (VNet)	Google Virtual Private Cloud (VPC)
Firewalls / Security ACLs	Security Groups (SGs) & Network ACLs (NACLs)	Network Security Groups (NSGs) & Azure Firewall	Firewall Rules & Cloud Firewall
Monitoring Tools	Amazon CloudWatch	Azure Monitor	Google Cloud Operations Suite (formerly Stackdriver)
CLI Tools	AWS Command Line Interface (AWS CLI)	Azure Command-Line Interface (Azure CLI)	Google Cloud SDK (gcloud)

Part 2: Comprehensive Analysis

2.1 Virtual Machines

Conceptual Model

- **AWS EC2: Instance-centric with marketplace ecosystem.** AWS treats VMs as disposable, fungible resources primarily launched from AMIs (Amazon Machine Images). The focus is on a vast marketplace of pre-configured community/partner AMIs and extreme specialization with dozens of instance families.
- **Azure VMs: Integrated Windows-first and enterprise-centric.** Azure's model is deeply integrated with Microsoft's ecosystem (Active Directory, Windows licensing, SQL Server). The conceptual model often feels like extending an on-premises data center, with strong hybrid cloud capabilities via Azure Arc.
- **GCP Compute Engine: Project-based, global, and automation-focused.** GCP thinks in terms of **Projects** as containers. A key conceptual difference is the **global VPC**; a network/subnet is not region-locked. VMs are seen as part of a larger, automated system leveraging Managed Instance Groups (MIGs) for true "cattle, not pets" management.

Ease of Use

- **AWS:** Moderate learning curve. The AWS Management Console can feel cluttered. The sheer number of options (instance types, purchasing models) is powerful but initially overwhelming. Launching a basic VM is straightforward.
- **Azure:** Very easy for Windows/AD admins. The Azure Portal is intuitive and well-organized. The "Create a resource" wizard is excellent for beginners. For Linux or non-Microsoft workloads, the experience is similar to AWS.
- **GCP: Simplest for getting started.** The Google Cloud Console is clean and uncluttered. Creating a VM is arguably the fastest and most straightforward process of the three. The Google cloud compute instances create command is incredibly succinct.

Terminology Differences (GCP vs. AWS/Azure)

- **Instance vs. Virtual Machine:** AWS/GCP say "Instance," Azure says "Virtual Machine."

- **Image Source:** AWS=**AMI**, Azure=**Image** (from Gallery/Marketplace), GCP=**Image** or **Instance Template** for scaling.
- **Scaling Group:** AWS=**Auto Scaling Group (ASG)**, Azure=**Virtual Machine Scale Set (VMSS)**, GCP=**Managed Instance Group (MIG)**.
- **Persistent Disk:** AWS=**EBS Volume**, Azure=**Managed Disk**, GCP=**Persistent Disk**. (Note: GCP Persistent Disks can be resized while attached and support live migration).
- **Big Confusion: GCP's "Global" VPC/Network.** An AWS/Azure engineer expects to define a subnet in a specific region/AZ. In GCP, you create a subnet with a *regional* scope (spanning all zones in that region) within a *global* VPC network. This is a fundamental architectural shift.

Strengths of GCP

1. **Sustained Use Discounts:** Automatic discounts (up to 30%) for instances running >25% of a month. Simpler than AWS Reserved Instances or Azure Reservations.
2. **Live Migration:** Maintenance events often don't reboot your VM; GCP migrates it live to other hardware. Huge for uptime.
3. **Global Load Balancing & Networking:** Seamless integration with global HTTP(S) and TCP/UDP load balancers that work natively with MIGs across regions.
4. **Simplicity & Speed:** Fastest VM creation time ("time to first byte") and the most intuitive console/CLI workflow.

Weaknesses or Gaps

1. **Instance Variety:** AWS has the broadest and most specialized instance catalog (e.g., for HPC, video encoding, etc.). GCP's portfolio, while robust, is smaller.
2. **Marketplace/AMI Ecosystem:** AWS's community AMI marketplace is vastly larger. GCP's equivalent (Cloud Marketplace) has fewer ready-to-go, third-party software images.
3. **Bare Metal:** AWS (EC2 Bare Metal) and Azure (BareMetal Instances) have more mature offerings for running directly on physical servers.
4. **Windows/Enterprise Focus:** Azure is dominant here. GCP supports Windows Server but doesn't have the same depth of integration with enterprise Microsoft tooling.

2.2 Object Storage

Conceptual Model

- **AWS S3: The definitive "bucket of objects".** S3 established model. It's a massive, durable, flat namespace where buckets are top-level containers. The focus is on universal compatibility, a massive ecosystem of tools, and granular control over every aspect (versioning, lifecycle, encryption, access logging).
- **Azure Blob Storage: Storage Account as the management unit.** The primary container is the **Storage Account** (with performance tiers: Standard/Premium). Within accounts, you create **Containers**, then **Blobs**. It's conceptually closer to a traditional file system hierarchy and is tightly integrated with other Azure data services.
- **GCP Cloud Storage: Consistency and simplicity as a unified object store.** GCP thinks of Cloud Storage as a single, unified service with a simple hierarchy: **Bucket -> Object**. Its killer conceptual feature is **strong global consistency** (list/read-after-write). It's designed to be the single backend for websites, data lakes, and backups.

Ease of Use

- **AWS:** Powerful but complex. The S3 console has accumulated many features, making it dense. Setting up proper permissions (bucket policies, ACLs) and configuring lifecycle rules has a steeper learning curve.
- **Azure:** Straightforward for basic use. The portal interface is clear. The concept of a "Storage Account" as a billing/management boundary adds an extra step but is logical. Access control can get complex when mixing Azure RBAC and blob-level ACLs.
- **GCP: Extremely easy for core functions.** Creating a bucket and uploading objects is intuitive. The permission model (IAM roles at project or bucket level) is simpler to grasp initially than S3's complex mix of policies and ACLs.

Terminology Differences (GCP vs. AWS/Azure)

- **Service Name:** AWS=**S3**, Azure=**Blob Storage**, GCP=**Cloud Storage**.
- **Primary Container:** AWS/GCP=**Bucket**, Azure=**Container** (nested inside a **Storage Account**).
- **Object Name:** AWS/GCP=**Key**, Azure=**Blob Name**.

- **Storage Tiers:** AWS=**Standard, Standard-IA, Glacier**, Azure=**Hot, Cool, Archive**, GCP=**Standard, Nearline, Coldline, Archive**. (Conceptually similar but performance/cost profiles differ).
- **Big Confusion: The "Storage Account" (Azure) has no direct equivalent.** In AWS/GCP, you just create a bucket. In Azure, you must first provision a Storage Account (which defines redundancy, performance tier, and location), *then* create containers within it.

Strengths of GCP

1. **Strong Global Consistency:** Once a write succeeds, any subsequent read, anywhere in the world, will get the latest data. This simplifies application logic dramatically compared to eventual consistency models.
2. **Unified Simpler API:** A single, clean API for all storage classes. No need to restore from "Archive" to a different tier before accessing; you just request the object (with higher latency/cost for archive).
3. **Performance:** Often demonstrates higher throughput, especially for sequential reads/writes of large objects.
4. **Multi-Regional Storage:** A unique class offering geo-redundancy across two or more regions for highest availability, not just within a single region.

Weaknesses or Gaps

1. **Tooling & Ecosystem:** The S3 API is the *de facto* standard. Countless third-party tools, libraries, and data platforms have built-in, optimized support for S3. GCS, while compatible via an interoperability layer, is not as natively supported.
2. **Storage Class Features:** AWS S3 Intelligent-Tiering (automatic cost optimization) is more mature. AWS also offers more granular retrieval options from Glacier (Expedited, Standard, Bulk).
3. **Integration with Analytics:** While Big Query has amazing integration with GCS, the broader AWS analytics ecosystem (Athena, Redshift Spectrum) has deeper, more performant integration with S3.

2.3 Monitoring Tools

Conceptual Model

- **AWS CloudWatch: Metrics and logs as the core of operational visibility.** CloudWatch is built around **Namespaces** for metrics and **Log Groups/Streams** for logs. It's a centralized service, but its components (Metrics, Logs, Alarms, Dashboards) can feel somewhat disjointed. The model is "collect everything here and build your views."
- **Azure Monitor: Unified data platform for everything telemetry.** Azure Monitor's model is a massive, integrated data pipeline. It ingests metrics, logs (from Azure Diagnostics/Agent), and application traces into a common **Log Analytics** backend. You then use **Kusto Query Language (KQL)** to query across all data types.
- **GCP Cloud Operations (formerly Stackdriver): Integrated observability for Google and hybrid clouds.** GCP thinks of monitoring as part of a broader **Operations** suite. It tightly integrates metrics, logs, traces (via Cloud Trace), and profiling (Cloud Profiler) with a focus on application performance and SRE practices (like SLO monitoring).

Ease of Use

- **AWS:** Can be fragmented. Navigating between CloudWatch Metrics, Logs Insights, and creating Alarms involves jumping between different sections of the console. Powerful but not always intuitive.
- **Azure: Powerful but has the steepest learning curve.** The Azure Monitor hub is comprehensive. The real power (and complexity) lies in **Log Analytics Workspace** and **KQL**. Writing effective KQL queries is a skill in itself.
- **GCP: Most cohesive and intuitive UI.** Cloud Monitoring and Cloud Logging feel like two views into the same system. The Metrics Explorer and Logs Explorer are very similar and easy to use. Creating dashboards and alerting policies follows a clear, guided workflow.

Terminology Differences (GCP vs. AWS/Azure)

- **Service Name:** AWS=**CloudWatch**, Azure=**Azure Monitor**, GCP=**Cloud Operations Suite** (product), **Cloud Monitoring** (metrics), **Cloud Logging** (logs).
- **Log Storage:** AWS=**Log Groups/Streams**, Azure=**Log Analytics Workspace/Tables**, GCP=**Log Buckets** (in Cloud Storage) and **Log Views**.

- **Query Language:** AWS=**CloudWatch Logs Insights**, Azure=**Kusto Query Language (KQL)**, GCP=**Logs Query Language (LogQL-like, but often just called "the query editor")**.
- **Alerting:** AWS=**CloudWatch Alarms**, Azure=**Alert Rules**, GCP=**Alerting Policies**.
- **Big Confusion: The "Workspace" concept.** In Azure, the **Log Analytics Workspace** is the central, billable container for all log data. AWS/GCP don't have a direct 1:1 equivalent; their log storage is more service-integrated.

Strengths of GCP

1. **Out-of-the-Box Visibility:** GCP services automatically export detailed, valuable metrics and logs with no configuration. The curated dashboards (e.g., for GKE, Compute Engine) are excellent.
2. **Built-in Application Performance Management (APM):** Cloud Trace (distributed tracing) and Cloud Profiler (CPU/memory profiling) are seamlessly integrated and often require just a library/agent addition to your app.
3. **SRE-Focused Tools:** Native support for **Service Level Objectives (SLOs)** and **Error Budgets** in the console. This is a conceptual advantage for teams adopting SRE practices.
4. **Simpler Logging Architecture:** No need to set up a "Log Analytics Workspace" or decide on retention per "Log Group." Logs go to a project-level sink with easy routing and retention policies.

Weaknesses or Gaps

1. **Third-Party & Hybrid Cloud Monitoring: Azure Monitor** excels here with Azure Arc, allowing consistent monitoring of on-premises servers. AWS also has strong hybrid agents. GCP's Operations Suite, while improving, started with a stronger GCP-native focus.
2. **Enterprise Dashboards & Reporting:** AWS CloudWatch Dashboards and Azure Monitor Workbooks are extremely powerful for building complex, interactive operational and business dashboards. GCP's dashboard tool, while improving, is less mature.
3. **Marketplace of Integrations:** The AWS and Azure ecosystems have a larger number of pre-built monitoring integrations for third-party software and SaaS products.

2.4 Web Application Migration

As for which AWS maps directly to GCP that would be the VMs the transitions from EC2 on AWS to GCE on google is almost the same process and would not require much if any relearning.

2.5 Key Concept differences (Major Processes)

As for the concepts I would say all concepts translate pretty well from GCP to AWS/Azure Other then the Naming schemes that would have to be familiarized most of all the concepts are near identical, The least familiar concepts from AWS to GCP would have to be their setup and distribution of VPC or Virtual Networks as most of GCP networking schemes are planned as global networks however in AWS by default they are usually Regional networks and thus the network infrastructure would have to be reworked.

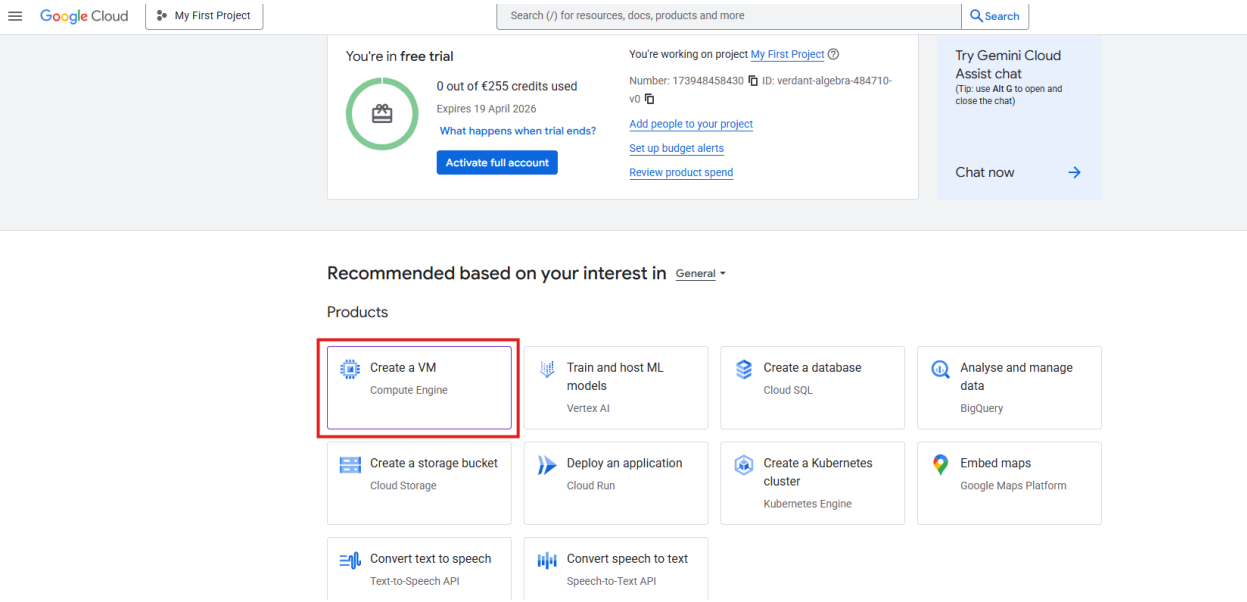
2.6 Different Conceptual Implementation (Minor Processes)

AWS Service	**GCP Service**	**Key Differences**
Lambda	Cloud Functions	Both serverless FaaS, but different runtime support and scaling models. GCP has longer timeout limits (up to 60 min).
EKS	Google Kubernetes Engine (GKE)	Both managed Kubernetes, but GKE is generally considered more mature and integrated (e.g., native multi-cluster features).
SQS/SNS	Cloud Pub/Sub	Different paradigm: Pub/Sub is a unified publish-subscribe service vs. AWS's separate queue (SQS) and topic (SNS) services.
CloudWatch	Cloud Monitoring	Both monitoring services, but GCP integrates tracing/profiling and has native SLO features.

2.7 Migration of web application

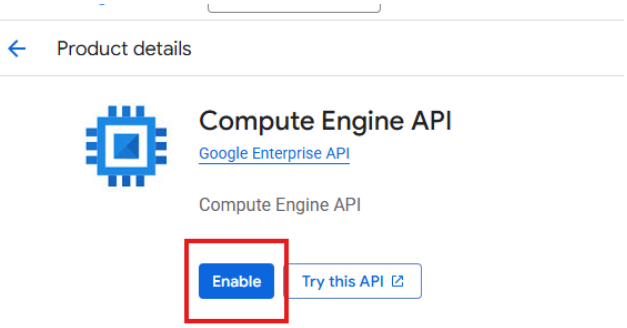
Step 1: Creation of Virtual Machine

On the GCP platform dashboard there will be a simple create VM or compute engine selection.



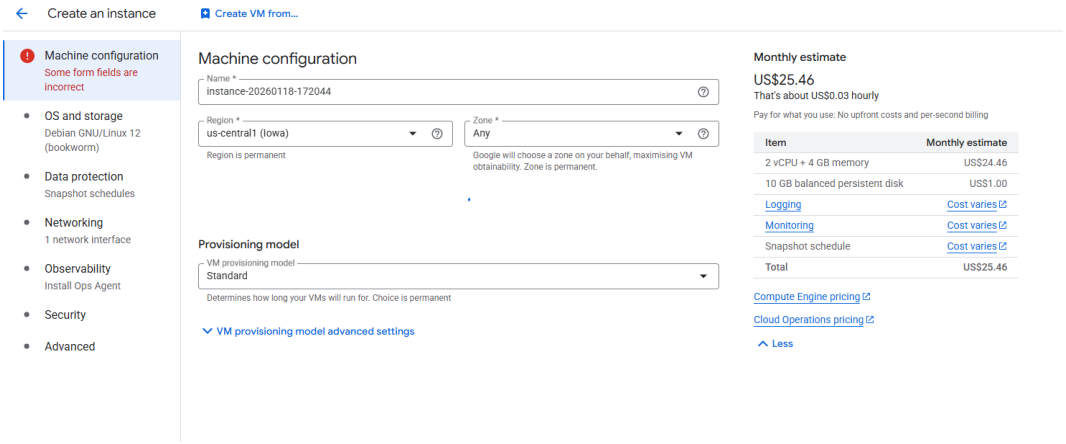
Step 2: Enable the resource

This will enable the service and prompt you with a wizard of your desired specifications for your webapp.

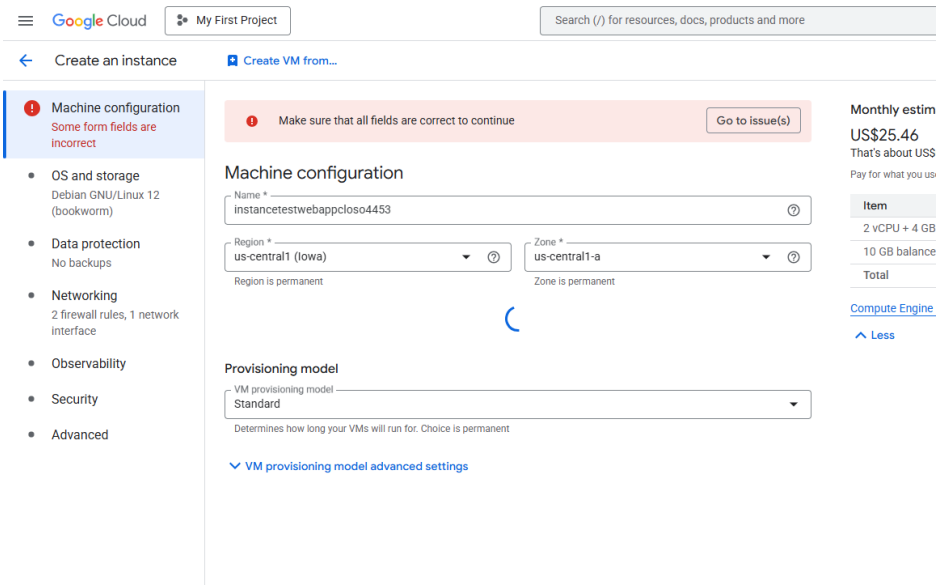


Step 3: Setup desired specifications

On the left-hand panel you can select any metric you want altered for your virtual machine and the preview cost will be shown on the right-hand side.



Then click the create VM button at the bottom left.



This will finalise the creation process and web migration VM should be created.

Part 3: DLP PDF Scanner.

3.0 Introduction

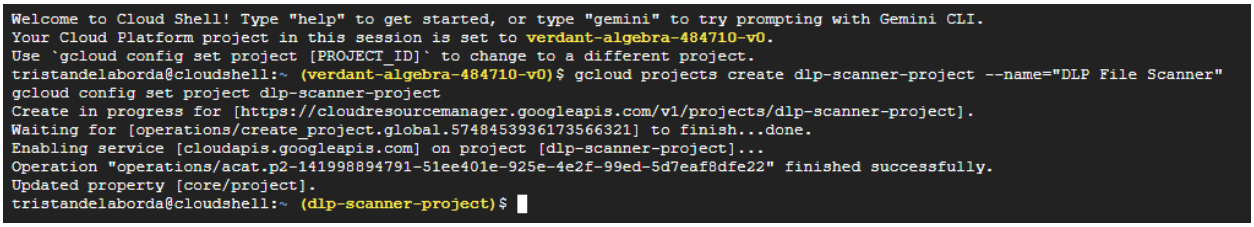
From the start we will be at the GCP dashboard and from here we will activate the Cloud shell so we can input some simple commands for our project initialization:

3.1 Creation of project

For the start of this part of the project we will be using the CLI and entering the following commands as it's simpler than using the GUI.

```
>gcloud projects create dlp-scanner-project --name="DLP File Scanner"
>gcloud config set project dlp-scanner-project
```

This will be the expected output:



This command will configure the start of our project named DLP File Scanner.

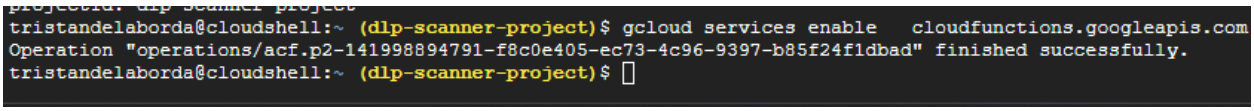
Additionally, for our project we will be needing to initialize some APIs on the GCP platform with the following commands:

3.2 Enabling Required APIs & Services

To enable the required Apis and services we will also be using the CLI to run the following commands enabling the services listed in the command.

```
>gcloud services enable \
  cloudfunctions.googleapis.com \
  cloudbuild.googleapis.com \
  storage.googleapis.com \
  dlp.googleapis.com \
  bigquery.googleapis.com \
  pubsub.googleapis.com
```

This will be the expected output:



3.3 Enabling Service Accounts & Permissions

To enable the account services and their respective permissions we will run the following code with the accounts named they will be used on.

```
# Create service account
gcloud iam service-accounts create dlp-scanner-sa \
  --display-name="DLP Scanner Service Account"

# Grant permissions for services
gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:dlp-scanner-sa@dlp-scanner-
project.iam.gserviceaccount.com" \
  --role="roles/dlp.user"

gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:dlp-scanner-sa@dlp-scanner-
project.iam.gserviceaccount.com" \
  --role="roles/storage.objectAdmin"

gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:dlp-scanner-sa@dlp-scanner-
project.iam.gserviceaccount.com" \
  --role="roles/bigquery.dataEditor"

gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:dlp-scanner-sa@dlp-scanner-
project.iam.gserviceaccount.com" \
  --role="roles/bigquery.jobUser"
```

3.4 Creation of Storage Bucket

Then we shall run the following commands to create the buckets we will be using for our data storage.

Create bucket with unique identifiers

```
BUCKET_NAME="dlp-scanner-uploads-$(date +%s)"
gsutil mb -l us-central1 gs://$BUCKET_NAME

# Create separate bucket for sensitive files (optional)
gsutil mb -l us-central1 gs://$BUCKET_NAME-sensitive
```

3.5 Creation Of BigQuery Dataset

We will be needing a dataset for the use of BIgQuery service on GCP so we will need to creation the dataset with our attributes so that the schema is followed when big query is carried out.

```
# Create dataset
bq mk --dataset dlp_scanner_project:dlp_results

# Create table schema file (dlp_results_schema.json)
cat > dlp_results_schema.json << EOF
[
  {"name": "file_name", "type": "STRING"},
  {"name": "bucket_name", "type": "STRING"},
  {"name": "scan_timestamp", "type": "TIMESTAMP"},
  {"name": "findings_count", "type": "INTEGER"},
  {"name": "info_type", "type": "STRING"},
  {"name": "likelihood", "type": "STRING"},
  {"name": "quoted_content", "type": "STRING"},
  {"name": "sensitive_data_found", "type": "BOOLEAN"},
  {"name": "action_taken", "type": "STRING"},
  {"name": "dlp_job_name", "type": "STRING"}
]
EOF

# Create table
bq mk --table \
  --schema dlp_results_schema.json \
  dlp_scanner_project.dlp_results.scan_results
```

We created this dataset for the storage to the Big query DB we will be needing this later for storage.

3.6 Cloud Functions

Now that we have the DB and the services enabled, we can move onto the VM and start adding our code that runs on the webapp.

So firstly, we will make a new directory for our workspace.

```
mkdir dlp-scanner-function
cd dlp-scanner-function
```

We will navigate into the workspace and from here we can make our two files, our txt requirements for python and a python script that we will be running on the VM.

```
cat > requirements.txt << EOF
google-cloud-dlp>=3.0.0
google-cloud-bigquery>=3.0.0
```

```
google-cloud-storage>=2.0.0
EOF
```

This is the Main.py for python contents:

```
cat > main.py << EOF
import base64
import json
import datetime
from google.cloud import dlp_v2
from google.cloud import bigquery
from google.cloud import storage

# Initialize clients
dlp_client = dlp_v2.DlpServiceClient()
bq_client = bigquery.Client()
storage_client = storage.Client()

# Configuration
PROJECT_ID = "dlp-scanner-project"
DATASET_ID = "dlp_results"
TABLE_ID = "scan_results"
SENSITIVE_BUCKET = "dlp-scanner-uploads-sensitive"  # Update with
your bucket

def insert_findings_to_bigquery(findings_data):
    """Insert DLP findings into BigQuery"""
    table_ref = f"{PROJECT_ID}.{DATASET_ID}.{TABLE_ID}"

    errors = bq_client.insert_rows_json(table_ref, findings_data)
    if errors:
        print(f"Errors inserting rows: {errors}")
    else:
        print(f"Successfully inserted {len(findings_data)} rows")

def move_to_sensitive_bucket(bucket_name, file_name):
    """Move sensitive files to quarantine bucket"""
    try:
        source_bucket = storage_client.bucket(bucket_name)
        destination_bucket =
storage_client.bucket(SENSITIVE_BUCKET)

        source_blob = source_bucket.blob(file_name)
        destination_blob = source_bucket.copy_blob(
            source_blob, destination_bucket, file_name
        )
        source_blob.delete()

        return f"moved to {SENSITIVE_BUCKET}"
    except Exception as e:
        return f"move failed: {str(e)}"

def dlp_scan_file(data, context):
    """Cloud Function triggered by Cloud Storage"""
    # Parse the event data
    if 'data' in data:
        file_data = base64.b64decode(data['data']).decode('utf-
8')
        event = json.loads(file_data)
    else:
        event = data

    # Get file info from event
    bucket_name = event['bucket']
    file_name = event['name']
```

```
    print(f"Scanning file: gs://{bucket_name}/{file_name}")

    # Skip already scanned or sensitive files
    if "sensitive" in bucket_name or
file_name.startswith("dlp_"):
        print(f"Skipping file from sensitive bucket or already
processed")
        return

    # Configure DLP job
    storage_config = {
        "cloud_storage_options": {
            "file_set": {"url":
f"gs://{bucket_name}/{file_name}"}
        }
    }

    # InfoTypes to detect (customize as needed)
    info_types = [
        {"name": "CREDIT_CARD_NUMBER"},
        {"name": "EMAIL_ADDRESS"},
        {"name": "PHONE_NUMBER"},
        {"name": "US_SOCIAL_SECURITY_NUMBER"},
        {"name": "PERSON_NAME"},
        {"name": "IBAN_CODE"},
        {"name": "IP_ADDRESS"}
    ]

    inspect_config = {
        "info_types": info_types,
        "min_likelihood": "POSSIBLE",
        "limits": {"max_findings_per_request": 100},
        "include_quote": True
    }

    # Create DLP job
    job_id = f"dlp-scan-
{datetime.datetime.now().strftime('%Y%m%d-%H%M%S')}"
    job_name = f"projects/{PROJECT_ID}/dlpJobs/{job_id}"

    dlp_job = {
        "inspect_config": inspect_config,
        "storage_config": storage_config,
        "job_id": job_id
    }

    # Submit the job (async call)
    try:
        print(f"Starting DLP job: {job_id}")
        parent = f"projects/{PROJECT_ID}"
        response = dlp_client.create_dlp_job(
            request={"parent": parent, "dlp_job": dlp_job}
        )

        # For simplicity, we'll use synchronous inspection
        # In production, you might want to use async jobs and
Pub/Sub notifications
        result = dlp_client.inspect_content(
            request={
                "parent": parent,
                "inspect_config": inspect_config,
                "item": {
                    "byte_item": {
                        "type_": "BYTES_TYPE_UNSPECIFIED",
                        "data":
storage_client.bucket(bucket_name)
```

```
                            .blob(file_name)
                            .download_as_bytes()
                    }
                }
            }
        )

        findings_data = []
        sensitive_found = False

        # Process findings
        for finding in result.result.findings:
            finding_info = {
                "file_name": file_name,
                "bucket_name": bucket_name,
                "scan_timestamp":
datetime.datetime.now().isoformat(),
                "findings_count": len(result.result.findings),
                "info_type": finding.info_type.name,
                "likelihood": finding.likelihood.name,
                "quoted_content": finding.quote[:500] if
finding.quote else "",
                "sensitive_data_found": True,
                "action_taken": "none",
                "dlp_job_name": job_name
            }
            findings_data.append(finding_info)
            sensitive_found = True

        # If no findings, log clean file
        if not sensitive_found:
            clean_record = {
                "file_name": file_name,
                "bucket_name": bucket_name,
                "scan_timestamp":
datetime.datetime.now().isoformat(),
                "findings_count": 0,
                "info_type": "NONE",
                "likelihood": "NONE",
                "quoted_content": "",
                "sensitive_data_found": False,
                "action_taken": "none",
                "dlp_job_name": job_name
            }
            findings_data.append(clean_record)
            print("No sensitive data found")

        # Insert findings to BigQuery
        if findings_data:
            insert_findings_to_bigquery(findings_data)

        # Take action if sensitive data found
        if sensitive_found:
            action = move_to_sensitive_bucket(bucket_name,
file_name)
            # Update the action in BigQuery
            for record in findings_data:
                record["action_taken"] = action
            # Re-insert with action (simplified - in production,
use UPDATE)
            insert_findings_to_bigquery(findings_data)
            print(f"Sensitive data found! File {action}")

    except Exception as e:
        print(f"Error scanning file: {str(e)}")
        # Log error to BigQuery
        error_record = [{
```

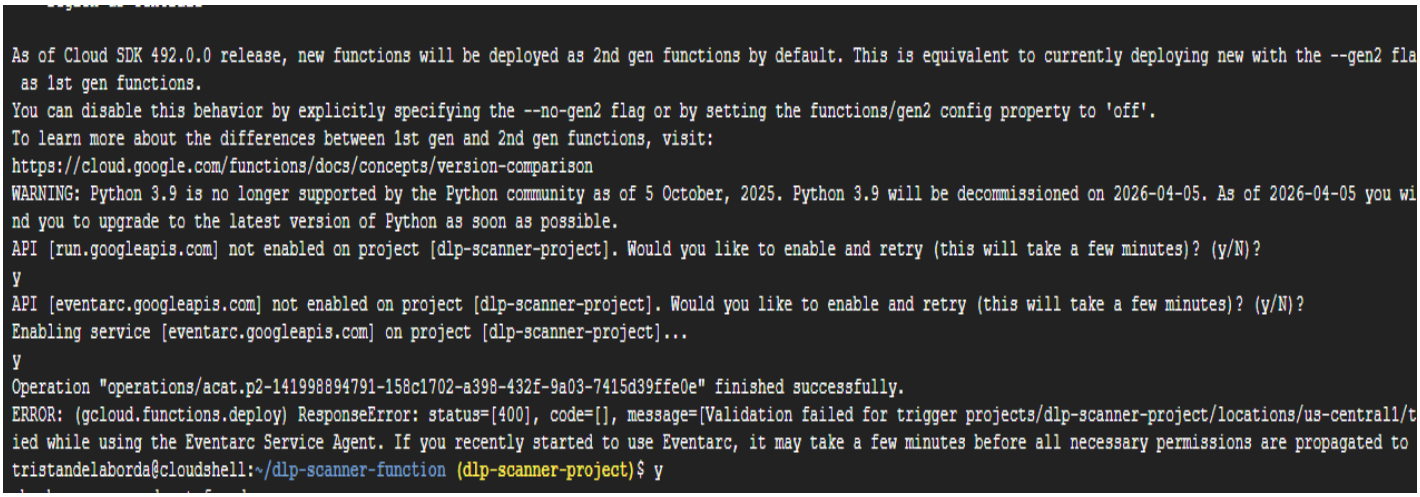
```
            "file_name": file_name,
            "bucket_name": bucket_name,
            "scan_timestamp":
datetime.datetime.now().isoformat(),
            "findings_count": -1,
            "info_type": "ERROR",
            "likelihood": "NONE",
            "quoted_content": str(e)[:500],
            "sensitive_data_found": False,
            "action_taken": "scan_failed",
            "dlp_job_name": "N/A"
        }]
        insert_findings_to_bigquery(error_record)
EOF
```

This file is long and so I tried my best to condense it using minimal line spacing where possible if some of the text seems unreadable it will be due to that.

Once we have those two files on the directory we will then move to enabling the cloud deployment on the VM. With the following command:

```
# Deploy the function (Python 3.9)
gcloud functions deploy dlp-file-scanner \
  --runtime python39 \
  --trigger-resource $BUCKET_NAME \
  --trigger-event google.storage.object.finalize \
  --service-account dlp-scanner-sa@dlp-scanner-
project.iam.gserviceaccount.com \
  --memory 512MB \
  --timeout 300s \
  --region us-central1
```

This will be the expected output if any prompts arise please simply type “Y” Then hit enter.



Operation should finish successfully this stipulates services on the VM are running and Python is running.

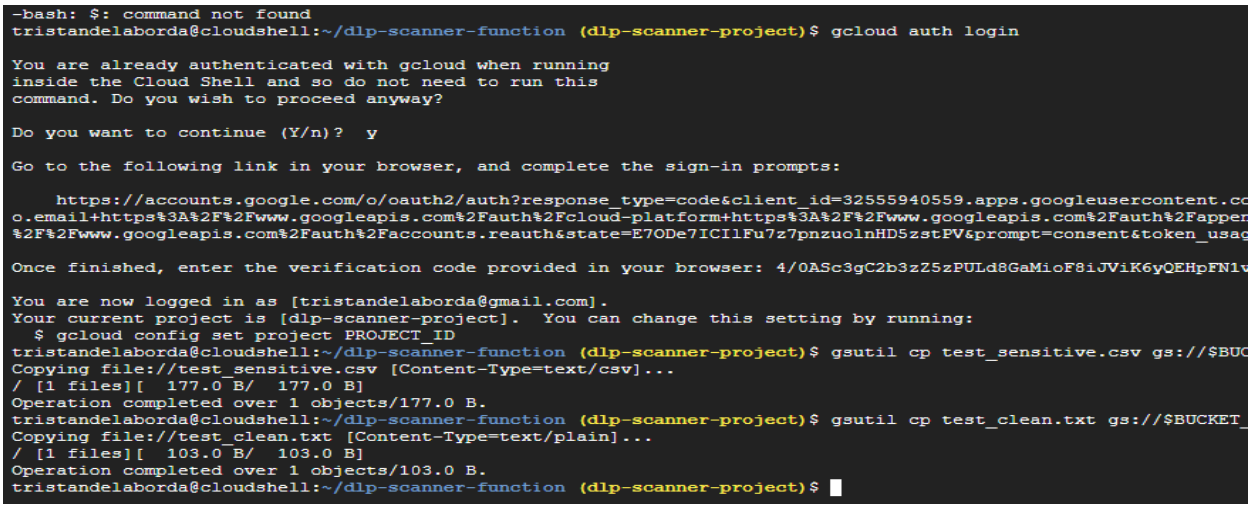
3.7 Testing Solution

Now we will Create a test file to simply text the functionality of our script on GCP.

```
# Create a test file with sensitive data
cat > test_sensitive.csv << EOF
Name,Email,Phone,Credit Card,SSN
John Doe,john@example.com,555-123-4567,4111-1111-1111-1111,123-
45-6789
Jane Smith,jane@company.com,555-987-6543,5500-0000-0000-0004,987-
65-4321
EOF
```

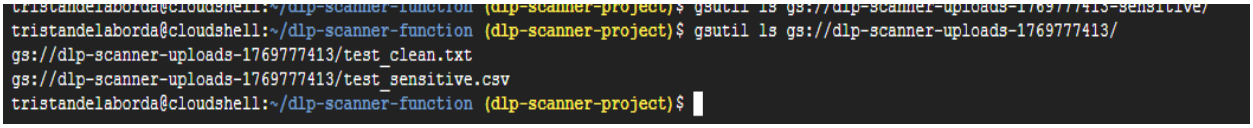
This will create two files in csv format one with its sensitive data and one as a control without any sensitive data, let's see if the application script will recognize this.

Now we will first need to login through our account on the cloud Cli client too to performa verification so that we can upload the files.



This should be done like this and the output should successfully verify that the upload has been successful.

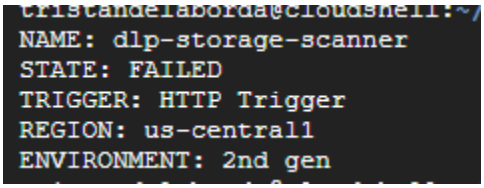
Using the following command, we can see the data is in the bucket we just uploaded:



After this we need to check if the function is working with

```
gcloud functions list --region us-central1
```

This will be the output



For some reason the state always failed no matter what I do, troubleshooting this problem proved to be more difficult than anticipated. It seems I don't have permissions do make the function run on the Cloud CLI.

First we need to give the storage permissions with the following commands:

```
# Storage service account email
STORAGE_SA="service-$PROJECT_NUMBER@gs-project-
accounts.iam.gserviceaccount.com"
echo "Storage Service Account: $STORAGE_SA"

# Grant Pub/Sub Publisher role
gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:$STORAGE_SA" \
  --role="roles/pubsub.publisher"
```

Then after this we need some permission Eventarc permissions also for some reason it keeps telling us we need permissions to run the function or it errors out during deployment.

```
# Eventarc service account
EVENTARC_SA="$PROJECT_NUMBER@gcp-sa-
eventarc.iam.gserviceaccount.com"
echo "Eventarc Service Account: $EVENTARC_SA"

# Grant roles to Eventarc
gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:$EVENTARC_SA" \
  --role="roles/pubsub.editor"

gcloud projects add-iam-policy-binding dlp-scanner-project \
  --member="serviceAccount:$EVENTARC_SA" \
  --role="roles/eventarc.eventReceiver"
```

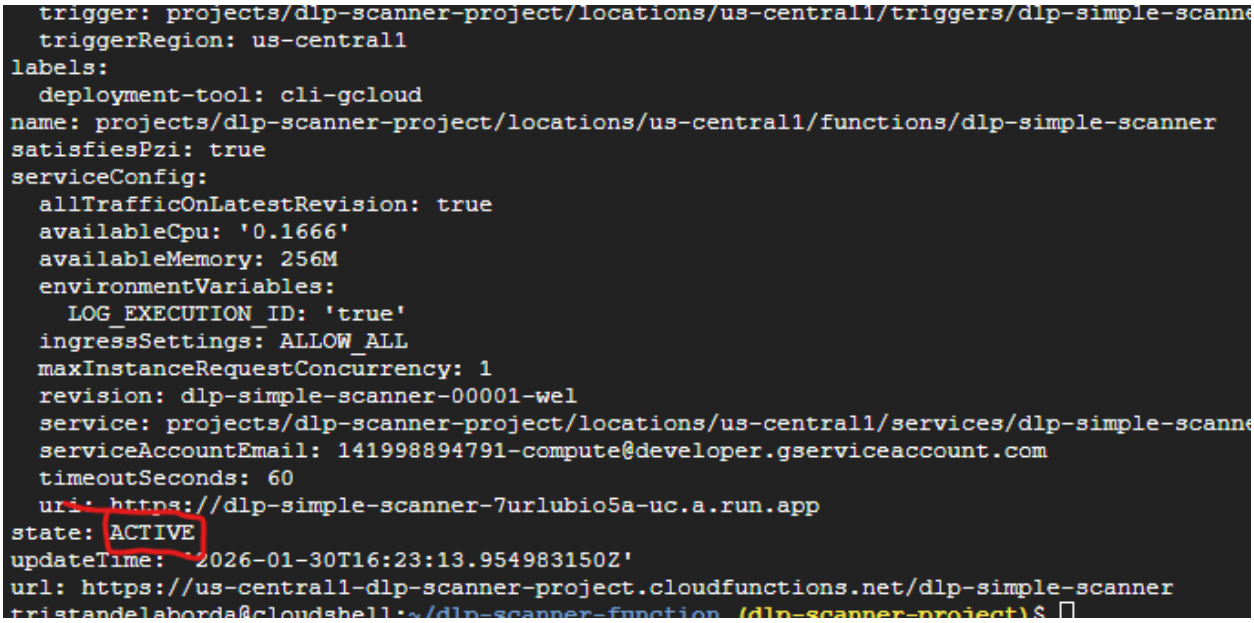
Now that we giving proper permissions we can make another function with the following commands that uses less permissions to operate this should be the expected output:

echo "3. Deploying legacy function..."

```
gcloud functions deploy dlp-simple-scanner \
  --runtime python39 \
  --trigger-bucket $BUCKET \
  --region us-central1 \
  --source . \
  --entry-point dlp_scan_file \
  --memory 256MB \
  --timeout 60s \
```

```
  --trigger-location us-central1 \
  --quiet
```

Expected output of function command:



Now we can test the functions works without data.

So, when we upload a new file using the following command:

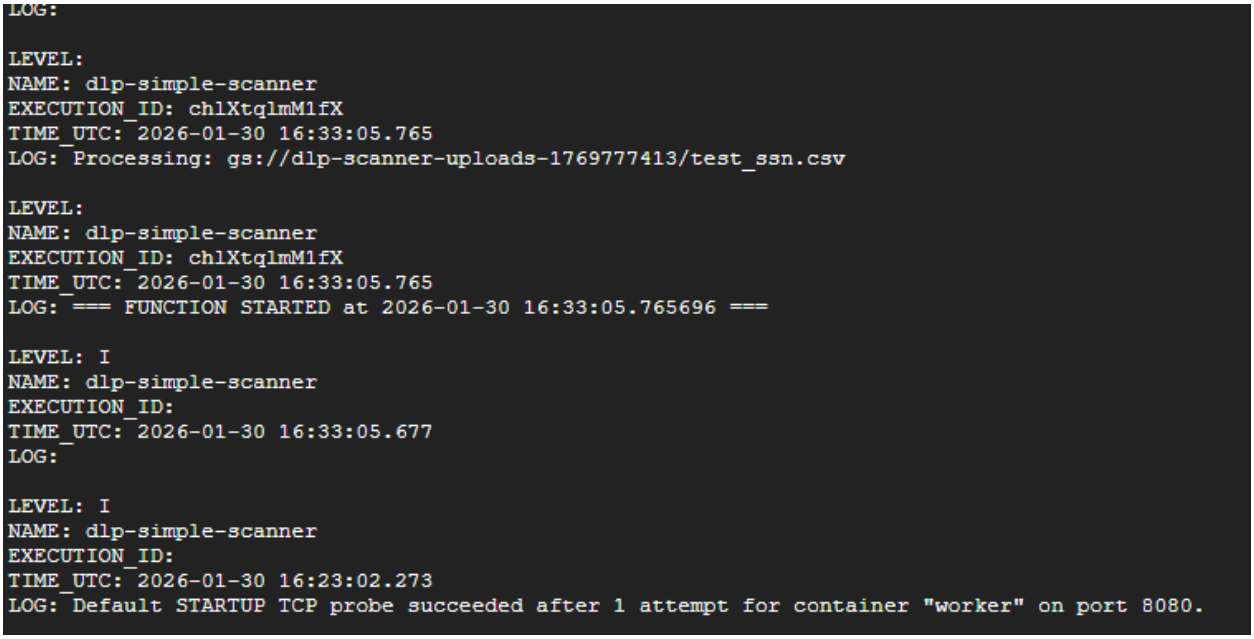
```
gsutil cp test_ssn.csv test_credit_card.txt test_clean.txt
gs://dlp-scanner-uploads-1769777413/
```

The function should trigger and alter the location based on the sensitive data of the file.

We can check the logs of the function to see if it was executed during our upload.

```
gsutil ls gs://dlp-scanner-uploads-1769777413/ | grep test
```

This command gives us the logs, and we are looking for a execution log to show that our function have scanned the data timestamped without current time.



With this output we can see the scanner did detect test_ssn.csv and started the function. This verifies the function now works on the GCP client.

4.0 Conclusion

As Google Cloud Platform is not my preferred platform to operate or create and implement solutions on I am slightly biased towards the bigger more fleshed out cloud providers. However, despite its limitations and somewhat clucky UI I feel it is still a modest competitor and can offer some value benefits other the other two still being able to be used for Cloud solutions such as the PDF Scanner. Overall it will suit the needs of smaller more lesser inclined business ventures and thus at a for forgiving cost.